



Amazon S3 — Complete Notes

Simple Storage Service · Object storage on AWS

A reader-friendly, all-in-one reference for Amazon S3.

1. 🪣 What is S3

Amazon S3 is an **object storage** service from AWS. It stores data as **objects** inside containers called **buckets**. Designed for **99.999999999% (11 nines)** durability and **99.99%** availability, with virtually unlimited capacity — you never provision disks. You pay only for what you store, the requests you make, and data transferred out.

💡 **Important:** S3 is *not* a file system and *not* block storage. It is flat object storage accessed over HTTP/HTTPS via a REST API. "Folders" are just a naming convention using `/` in the key.

Common uses: backups, static website hosting, data lakes, media storage, software distribution, log storage, big-data analytics, archival.

2. 🧩 Core Concepts

Concept	Description
Bucket	Top-level container for objects. Name is globally unique across all AWS accounts. Lives in one region .
Object	The data + metadata. Made of Key, Value, Version ID, Metadata, ACL info. Size 0 B → 5 TB .
Key	Full unique name of an object, e.g. <code>images/2024/photo.jpg</code> .
Prefix	Start of a key, used to group/filter objects (simulates folders).
Delimiter	Usually <code>/</code> , used by list ops to roll keys into folder-like groups.
Region	Geographic location where the bucket physically lives — chosen for latency, cost, compliance.

Bucket naming rules: 3–63 chars · lowercase, numbers, hyphens, dots · must start/end with letter or number · no uppercase, no underscores · cannot look like an IP address.

Upload limits: single `PUT` max **5 GB** → above that you **must** use multipart upload (recommended for anything > 100 MB).

3. Storage Classes

All classes offer the same **11 nines** durability (*except One Zone, which is single-AZ*). They trade off cost vs. access speed vs. retrieval fees.

Class	Best for	Access speed	Min. duration	Notes
Standard	Frequently accessed data	Milliseconds	—	Default. ≥3 AZs. No retrieval fee.
Intelligent-Tiering	Unknown / changing patterns	Milliseconds	—	Auto-moves between tiers. Small monitoring fee, no retrieval fee.
Standard-IA	Infrequent, rapid access	Milliseconds	30 days	Cheaper storage + per-GB retrieval fee.
One Zone-IA	Re-creatable / secondary data	Milliseconds	30 days	Single AZ → ~20% cheaper, lower resilience.
Glacier Instant Retrieval	Archive, instant access	Milliseconds	90 days	Cheap storage, higher retrieval cost.
Glacier Flexible Retrieval	Backups / DR archives	Minutes–hours	90 days	Expedited (1–5 min), Standard (3–5 h), Bulk (5–12 h).
Glacier Deep Archive	Long-term compliance	12–48 hours	180 days	Lowest cost in AWS.
S3 on Outposts	On-prem data residency	Local	—	S3 on AWS Outposts hardware.

4. Durability, Availability, Consistency

- **Durability** — **99.999999999% (11 nines)**. Store 10M objects and expect to lose ~1 every 10,000 years. Achieved via redundant storage across devices and (for most classes) multiple AZs.
- **Availability** — **varies**: Standard **99.99%**, Standard-IA **99.9%**, One Zone-IA **99.5%**.
- **Consistency** — **strong read-after-write** for all operations (since Dec 2020). A read right after a write/overwrite/delete returns the latest data.  No more eventual-consistency surprises.

5. Versioning

- Keeps **multiple versions** of an object in the same bucket — protects against accidental deletes/overwrites.
- States: **Unversioned** (default) · **Enabled** · **Suspended**. Once enabled it can only be *suspended*, never fully disabled.
- Each version gets a unique **Version ID** (`null` for pre-versioning objects).
- A **DELETE** adds a **delete marker** — it does *not* erase data; the prior version stays recoverable. To truly delete, delete a specific version ID.

- Increases storage cost (every version is stored) → pair with **lifecycle rules** to expire old versions.
- **MFA Delete**: optional protection requiring an MFA token to delete versions or change versioning state (root-only, via CLI).

6. 🔄 Lifecycle Management

Rules that automatically **transition** or **expire** objects to save cost.

- **Transition actions** — move to a cheaper class after N days (*e.g. Standard → Standard-IA after 30 days → Glacier after 90 days*).
- **Expiration actions** — delete objects after N days, delete old versions, abort incomplete multipart uploads, remove expired delete markers.

Rules can target the whole bucket, a prefix, or object **tags**. Great for log retention, compliance, and cost optimization.

⚠️ Cannot transition to IA before 30 days; very small objects may not be worth moving.

7. 🛡️ Security & Access Control

Everything in S3 is private by default. Only the bucket owner has access.

Layer	What it does
IAM Policies	Identity-based — what <i>your</i> users/roles can do.
Bucket Policies	JSON resource policy on a bucket; cross-account access, enforce HTTPS/encryption, restrict by IP. Most common bucket-wide control.
ACLs	Legacy object/bucket grants. AWS recommends disabling them ("Bucket owner enforced").
Block Public Access	Account/bucket safety switch (4 settings). ON by default — prevents accidental public exposure.
Access Points	Named endpoints with their own policies, for managing shared buckets/data lakes at scale.
Presigned URLs	Time-limited URL granting access to one object without making it public.
VPC Endpoint	Private access to S3 from a VPC without going over the public internet.

8. 🔑 Encryption

In transit: use HTTPS/TLS — can be enforced via bucket policy (`aws:SecureTransport`).

At rest (server-side encryption, SSE):

Type	Who manages keys	Notes
SSE-S3 (AES-256)	AWS / S3	Default since Jan 2023 — all new objects encrypted automatically.
SSE-KMS	AWS KMS	Audit trail (CloudTrail), key rotation, fine-grained access. Slight cost + API limits.
SSE-C	Customer (per request)	AWS doesn't store the key.
Client-side	You, before upload	AWS never sees plaintext.

DSSE-KMS: dual-layer KMS encryption for high-compliance needs.

9. 🚀 Data Transfer & Performance

- **Multipart Upload** — split a large object into parallel parts. Required > 5 GB, recommended > 100 MB. Up to **10,000 parts**, each ≤ 5 GB. Resumable. *Abort incomplete uploads via lifecycle rule to avoid orphan charges.*
- **Transfer Acceleration** — uses CloudFront edge locations + AWS backbone to speed long-distance transfers (extra cost).
- **Byte-Range Fetches** — download just part of an object via the **Range** header; enables parallelism.
- **Performance** — scales to **≥ 3,500 writes** and **≥ 5,500 reads per second per prefix**. Spread keys across prefixes for more parallel throughput.
- **S3 Select** — pull a subset of data using SQL (CSV/JSON/Parquet) so you transfer less.

10. 🌐 Replication

Automatically copy objects to another bucket. **Requires versioning** on source *and* destination.

Type	Scope	Use cases
CRR (Cross-Region)	Different region	Compliance, lower latency for distant users, disaster recovery.
SRR (Same-Region)	Same region	Log aggregation, prod/test sync, cross-account.

- Can replicate to a different **account** and a different **storage class**.
- Only replicates **new** objects after the rule is enabled (use **Batch Replication** for existing ones).
- **Replication Time Control (RTC)** offers a 15-minute SLA.

11. 🌐 Static Website Hosting

- Host static sites (HTML/CSS/JS/images) — **no server-side code**.
- Enable *Static website hosting*, set index + error documents.
- Bucket must allow public read (or front it with **CloudFront**).
- Endpoint: `bucket-name.s3-website-region.amazonaws.com`
- For HTTPS + custom domain + caching → put **CloudFront** in front.

12. 🛎 Event Notifications & Integration

- S3 emits events on actions (object created, deleted, restored...).
- **Targets:** SNS (notify), SQS (queue), Lambda (run code), EventBridge (advanced routing/filtering).
- **Common pattern:** upload object → trigger Lambda → process file (resize image, ingest data, etc.).

13. 📊 Monitoring, Logging, Analytics

Tool	Purpose
Server Access Logging	Detailed request records, stored in another bucket.
CloudTrail	Audit S3 API calls (management + data events).
CloudWatch	Metrics: bucket size, object count, request metrics.
Storage Lens	Org-wide usage/activity visibility + cost recommendations.
S3 Inventory	Scheduled report (CSV/ORC/Parquet) of objects + metadata.
Storage Class Analysis	Analyzes access patterns to recommend moving to IA.

14. 🛒 Other Features

- **Object Lock (WORM)** — Write Once Read Many. Prevents delete/overwrite for a retention period or legal hold. Modes: **Governance** (override with permission) and **Compliance** (no one, not even root). Requires versioning.
- **Object Tags** — up to 10 key-value tags per object; drive lifecycle, access control, cost allocation, filtering.
- **Requester Pays** — the requester (not owner) pays for requests + transfer; for sharing large datasets.
- **S3 Batch Operations** — bulk actions on billions of objects (copy, tag, restore, invoke Lambda).
- **S3 Object Lambda** — transform data on retrieval (redact, resize, convert) without storing copies.
- **Mountpoint for S3** — mount a bucket as a local filesystem for read-heavy workloads.

15. 💰 Pricing

You pay for:

1. **Storage** — per GB/month, by class.
2. **Requests** — per 1,000 (PUT, GET, LIST...), varies by class.
3. **Data transfer OUT** — to the internet (*IN is free; S3 ↔ EC2 same-region is free*).
4. **Retrieval fees** — for IA & Glacier classes (per GB retrieved).
5. **Management features** — Intelligent-Tiering monitoring, inventory, analytics, replication.

6. **Early-delete fees** — deleting before a class's minimum duration.

 **Cost tips:** use lifecycle rules → cheaper classes · Intelligent-Tiering for unknown patterns · delete old versions + incomplete multipart uploads · keep compute in the same region · use a VPC Gateway Endpoint to dodge NAT/transfer charges.

16. **Common CLI Commands**

```
aws s3 mb s3://my-bucket           # make/create bucket
aws s3 rb s3://my-bucket           # remove bucket
aws s3 ls                           # list buckets
aws s3 ls s3://my-bucket/          # list objects
aws s3 cp file.txt s3://my-bucket/  # upload a file
aws s3 cp s3://my-bucket/file.txt . # download a file
aws s3 sync ./localdir s3://my-bucket/ # sync a folder
aws s3 rm s3://my-bucket/file.txt   # delete an object
aws s3 presign s3://my-bucket/file.txt # generate presigned URL
```

Low-level API uses `aws s3api ...` for fine control (versioning, policies, multipart, etc.).

17. **Key Limits & Numbers**

Item	Value
Max object size	5 TB
Max single PUT upload	5 GB (multipart above this)
Max multipart parts	10,000 (each ≤ 5 GB)
Bucket name	3–63 chars, globally unique, lowercase
Buckets per account	100 default (up to 1,000 on request)
Durability	11 nines (99.99999999%)
Standard availability	99.99%
Object tags	10 max per object
User-defined metadata	2 KB limit
Request rate per prefix	3,500 writes / 5,500 reads per second
Min storage duration	IA 30d · Glacier 90d · Deep Archive 180d

18. 🚧 Ways to Connect to S3

Everything ultimately calls the same REST API — only the entry point differs. Almost every method needs valid **credentials + IAM permissions**.

A. Credentials first (authentication)

- **IAM user access keys** — Access Key ID + Secret. Used by CLI/SDK. Avoid long-lived keys.
- **IAM roles** (*preferred*) — EC2 instance role, Lambda exec role, ECS task role, EKS IRSA. Temporary creds, nothing stored in code.
- **STS temporary credentials** — short-lived tokens via AssumeRole / federation / web identity. Best for cross-account / temporary access.
- **Root account** — never for normal access.

B. AWS Management Console (web UI)

Browser-based — create buckets, upload/download, set policies. Good for manual/admin tasks.

C. AWS CLI

`aws s3 ...` (high-level) and `aws s3api ...` (low-level). Configure with `aws configure` or named profiles (`--profile myprofile`). Great for scripts/automation.

D. AWS SDKs (in code)

Python (**boto3**), JS/Node (**@aws-sdk/client-s3**), Java, **Go (aws-sdk-go-v2)**, .NET, Ruby, PHP, C++, Rust... SDK handles SigV4 signing, retries, multipart, pagination.

```
# Python (boto3)
import boto3
s3 = boto3.client("s3")
s3.upload_file("local.txt", "my-bucket", "key.txt")
s3.download_file("my-bucket", "key.txt", "local.txt")
```

```
// Go (aws-sdk-go-v2)
cfg, _ := config.LoadDefaultConfig(ctx)
client := s3.NewFromConfig(cfg)
client.PutObject(ctx, &s3.PutObjectInput{ /* ... */ })
```

E. REST API directly (HTTP)

Raw HTTPS requests signed with **AWS Signature v4**.

```
https://my-bucket.s3.region.amazonaws.com/key      (virtual-hosted)
https://s3.region.amazonaws.com/my-bucket/key     (path-style, legacy)
```

Usually let an SDK/CLI do the signing.

F. Presigned URLs

Time-limited signed URL to GET or PUT a specific object **without** AWS credentials. Common for browser uploads/temporary downloads.

```
aws s3 presign s3://bucket/key --expires-in 3600
```

G. Mount as a filesystem

- **Mountpoint for Amazon S3** — official AWS client; mounts a bucket as a folder (read-heavy / sequential).
- **s3fs-fuse** — third-party FUSE mount (slower, not a true FS).
- **AWS Storage Gateway (File Gateway)** — S3 via NFS/SMB with local caching for on-prem apps.

H. Private network connectivity

- **VPC Gateway Endpoint** — reach S3 privately from a VPC, no IGW/NAT, no data leaves AWS. **Free**.
- **VPC Interface Endpoint (PrivateLink)** — private IP in your subnet (has cost; on-prem-to-VPC).
- **Direct Connect / VPN** — connect on-prem networks to AWS, then reach S3.

I. Third-party / GUI tools

Cyberduck, S3 Browser, WinSCP, Rclone, Transmit — speak the S3 API using your keys. Good for manual transfers and cross-cloud sync (rclone).

J. Through other AWS services (S3 as backend)

CloudFront (CDN), **Athena** (SQL over S3), **Glue**, **EMR/Spark**, **Redshift Spectrum**, **Lambda** triggers, **DataSync** (bulk migration), **Transfer Family** (SFTP/FTPS/FTP front end).

K. S3-compatible access

The S3 API is a de-facto standard — point the SDK/CLI at a custom **endpoint URL** to talk to MinIO, Wasabi, Ceph, etc.

Quick pick guide

Need	Use
One-off manual task	Console or GUI tool
Scripting / automation	AWS CLI
Application code	AWS SDK (boto3, aws-sdk...)
Let a user upload/download	Presigned URL
App inside a VPC	IAM role + VPC Gateway Endpoint
Legacy app needs a filesystem	Mountpoint / Storage Gateway
Serve files to the web	CloudFront in front of S3

19. Quick Mental Model

- **S3 = unlimited buckets of objects**, accessed over HTTP, extremely durable.
- Pick a **storage class** by how often you read data and how fast you need it back.
- Everything is **private by default** — open access deliberately via IAM + bucket policy; keep **Block Public Access ON**.

- Turn on **Versioning + Lifecycle** rules to stay safe and cheap.
- **Encrypt** in transit (HTTPS) and at rest (SSE on by default).
- Use **Multipart** for big files, **Transfer Acceleration / CloudFront** for speed.
- Use **Events + Lambda** to make storage reactive.
- Use **Replication** for DR/compliance, **Object Lock** for WORM compliance.
- **Monitor** with CloudWatch, CloudTrail, Storage Lens; optimize cost relentlessly.

20. ? Common Interview Questions

Rapid-fire questions interviewers ask about S3:

- **Q: What is S3's consistency model?** — Strong read-after-write consistency for **all** operations (PUT, overwrite, DELETE) since Dec 2020. A read right after a write returns the latest data.
- **Q: S3 vs EBS vs EFS?** — S3 = object storage over HTTP (no filesystem). EBS = block storage for a single EC2 (one AZ). EFS = shared NFS filesystem across many instances.
- **Q: How do you secure an S3 bucket?** — Block Public Access ON, least-privilege IAM + bucket policy, default SSE encryption, enforce HTTPS (`aws:SecureTransport`), VPC endpoint for private access, enable versioning + logging.
- **Q: Why use versioning?** — Protects against accidental delete/overwrite; a DELETE adds a *delete marker* (recoverable) rather than erasing data.
- **Q: Object size limits?** — Up to 5 TB; single PUT max 5 GB → use **multipart upload** above that (recommended > 100 MB).
- **Q: When Glacier vs Standard-IA?** — IA for infrequent but instant access; Glacier (Instant/Flexible/Deep Archive) for archival where retrieval time/cost is acceptable.
- **Q: What is a pre-signed URL?** — A time-limited signed URL granting temporary access to one object without making it public.
- **Q: How to reduce S3 cost?** — Lifecycle rules to cheaper classes, Intelligent-Tiering for unknown patterns, delete old versions + incomplete multipart uploads, keep compute in-region.

21. 🎯 Detailed Interview Q&A and When to Use

Scenario-style interview questions with model answers and **when to use** guidance. Aimed at ~2 years of hands-on experience.

Basics

- 1. What is S3 / what is object storage?** Massively durable object store accessed over an HTTP API. Object storage = a flat key-space of immutable blobs with metadata, no in-place edits, scales infinitely — versus block (raw volume, EBS) or file (POSIX hierarchy, EFS) storage. **When to use:** S3 → objects, backups, static assets, data lake, logs, anything via API. EBS → single-instance block volume. EFS → shared POSIX filesystem across many instances.
- 2. Object structure; is S3 a filesystem?** Object = key (full path string) + value (bytes) + metadata + version ID. No real folders — the "folders" in the console are just a UI grouping on the `/` delimiter. The bucket is a flat namespace.
- 3. Storage classes + tradeoff?** Standard (hot). Standard-IA / One Zone-IA (cheaper storage, per-GB retrieval fee, min duration). Intelligent-Tiering (auto-moves by access pattern). Glacier Instant / Flexible / Deep Archive (cheapest storage, retrieval latency ms→hours). Lower storage cost buys higher retrieval cost/latency. **When to use:** Standard → frequently accessed/hot. Standard-IA → infrequent but needs instant access. One Zone-IA → infrequent + reproducible (OK to lose). Intelligent-Tiering →

unknown/changing access pattern. Glacier Instant → archive with rare instant access. Glacier Flexible/Deep Archive → cold/compliance, minutes-to-hours retrieval is fine.

4. Durability vs availability; what is "11 nines"? Durability = will the object survive (won't be lost/corrupted) — S3 Standard is 99.999999999% (11 nines). Availability = can you access it right now — Standard is 99.99%. Eleven nines refers to durability, not uptime.

5. Make an object/bucket public — why is that the wrong instinct? You can via ACL/bucket policy, but public buckets are the #1 cause of data leaks. Modern pattern: keep Block Public Access on and serve via CloudFront + OAC or pre-signed URLs. **When to use:** Public → effectively never; for public static sites use CloudFront + a private bucket. Direct public-read only for a deliberately open dataset.

Intermediate

6. The 4 access-control layers — who wins on conflict? IAM policies (identity), bucket policies (resource), ACLs (legacy per-object), and Block Public Access (account/bucket override). Effective permission = union of Allows minus any explicit Deny; an explicit Deny anywhere wins, and BPA overrides anything that would grant public access. **When to use:** IAM policy → control by who (users/roles in your account). Bucket policy → control by resource (cross-account, broad bucket rules). ACL → avoid (legacy). Block Public Access → always on.

7. Versioning — solves what, costs what? Keeps every version, so overwrites/deletes are recoverable (a delete just adds a delete marker). New problem: you pay for every version forever — without lifecycle rules to expire old versions, cost balloons silently. **When to use:** Enable → important/irreplaceable data, ransomware/accident protection, compliance. Skip (or aggressively expire) → high-churn temp/log buckets where cost matters.

8. Lifecycle policy — design for logs (hot 30d, cheap 1yr, then delete). Day 0–30 Standard → transition to Standard-IA or Glacier at day 30 → expire at day 395. Add rules to expire noncurrent versions and abort incomplete multipart uploads. **When to use:** Any bucket with predictable aging data (logs, backups) — transition to cheaper tiers + expire; also to clean noncurrent versions and incomplete multipart uploads.

9. SSE-S3 vs SSE-KMS vs SSE-C; when does KMS bite? SSE-S3 = AWS-managed keys, zero config. SSE-KMS = KMS-managed keys, audit trail + key access control. SSE-C = you supply the key per request. KMS bites on cost and request throttling — every GET/PUT calls KMS (use bucket keys to cut calls). **When to use:** SSE-S3 → default, no key-management needs. SSE-KMS → need audit trail / key access control / compliance. SSE-C → you must hold/manage keys yourself off-AWS.

10. Versioning + MFA Delete / Object Lock vs ransomware? Versioning means a malicious overwrite/delete doesn't destroy data. MFA Delete requires an MFA token to permanently delete versions, so compromised credentials can't wipe history. **When to use:** MFA Delete → protect against credential compromise wiping versions. Object Lock (compliance) → legal/regulatory WORM (SEC 17a-4, audit retention).

11. Pre-signed URL; direct browser→S3 upload? A time-limited signed URL granting temporary access to a specific operation/object using your credentials. Client asks your backend → backend returns a pre-signed PUT URL → client uploads directly to S3; bytes never touch your server. **When to use:** Let a client upload/download directly without your server proxying bytes, time-limited, no public bucket.

12. Strong read-after-write consistency — what was the old behavior? S3 is now strongly consistent for all operations. Previously, overwrite PUTs and LISTs were eventually consistent — a read right after an overwrite could return stale data, which broke write-then-read data pipelines.

Advanced & Scenario

13. Scenario — 503 SLOW DOWN under heavy load? You're exceeding S3's per-prefix request rate (~3,500 writes / 5,500 reads per prefix per second). Spread keys across more prefixes so requests parallelize across partitions, and add retries with exponential backoff/jitter on the app side. High-cardinality prefix designs avoid hotspotting.

14. Scenario — GET returns 403 not 404; what does that tell you? 404 = object genuinely missing (and you have ListBucket). 403 = you exist but aren't authorized — and if you lack `s3:ListBucket`, S3 returns 403 even for missing keys to avoid leaking existence. So 403-instead-of-404 points at a permissions gap (missing ListBucket or a Deny), not a missing file.

- 15. Scenario — S3 bill tripled, data volume flat?** Usually not storage. Investigate request costs (a hot GET/PUT loop or KMS calls), data transfer/egress (esp. cross-region or to internet), versioning piling up noncurrent versions, incomplete multipart uploads, or a broken lifecycle. Use S3 Storage Lens + Cost Explorer + CloudWatch request metrics.
- 16. CRR vs Multi-Region Access Points?** CRR = async replication of objects to a bucket in another region. MRAP = a single global endpoint that routes requests to the nearest/healthy bucket. **When to use:** CRR → you need copies in another region (DR, compliance, data residency, locality). MRAP → you need clients transparently routed to the nearest/healthy copy.
- 17. Serve global static assets with a locked origin — design + OAC vs OAI.** CloudFront in front of a private bucket, access via Origin Access Control (OAC); bucket policy allows only the distribution, Block Public Access stays on. OAC replaces the older OAI — it supports SSE-KMS, all regions (SigV4), and PUT/POST. **When to use:** OAC → always for new CloudFront→S3. OAI → legacy only.
- 18. Enforce "no public buckets" across hundreds of accounts?** Account-level Block Public Access plus an AWS Organizations SCP that denies the API calls which would make buckets public. **When to use:** SCP → org-wide guardrail across many accounts that no account admin can override.
- 19. Why is `aws s3 sync` of millions of small files slow; how to speed a migration?** Each tiny object is a separate API round-trip; latency, not bandwidth, dominates. Increase concurrency (`max_concurrent_requests`), parallelize by prefix, use Transfer Acceleration or DataSync, and aggregate tiny files where possible. **When to use:** `s3 sync` → small/occasional. Transfer Acceleration → long-distance uploads over public internet. DataSync → large one-time/scheduled migrations (millions of files, on-prem→S3).
- 20. Object Lock — governance vs compliance; the catch?** WORM — prevents delete/overwrite until a retention date. Governance = privileged users can override. Compliance = no one, not even root, can delete before retention expires; it's irreversible, so a mistake locks storage (and cost) for the full retention period. **When to use:** Governance → protect data but allow privileged override (most internal cases). Compliance → regulatory; use only when legally required.
- 21. Cross-cutting — S3 cost leaks and security fails to hunt first.** Cost leaks: noncurrent versions with no expiry, cross-region transfer/egress, KMS request charges, incomplete multipart uploads, wrong storage class for the access pattern. Security fails: public bucket / public ACL, no default encryption, no Block Public Access, HTTP allowed (missing `aws:SecureTransport` deny), no access logging.